

JAVA COLLECTION FRAMEWORK – 12 PROGRAMS

Complete Practical Record

Each program contains Aim, Algorithm, UML Class Diagram, Source Code, Execution Commands, Output Screenshot and Result.

Program 1: Collection Methods Demo

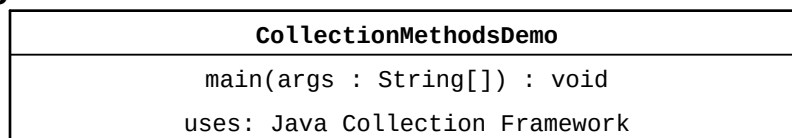
Aim / Objective

To demonstrate commonly used methods of the Collection interface using an ArrayList.

Algorithm / Logic

1. Create a Collection of String values using ArrayList.
2. Add elements using add() and addAll().
3. Check elements using contains() and containsAll().
4. Display size and empty status.
5. Remove elements using remove() and removeAll().
6. Traverse the collection using Iterator.
7. Clear the collection and check isEmpty().

UML Class Diagram



Source Code

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collection;
import java.util.Iterator;

class CollectionMethodsDemo {

    public static void main(String[] args) {

        // Creating a Collection
        Collection<String> students = new ArrayList<>();

        // add(E e)
        students.add("Rahul");
        students.add("Priya");
        students.add("Arjun");

        System.out.println("After add(): " + students);

        // addAll(Collection c)
        Collection<String> newStudents =
            Arrays.asList("Anu", "Kiran");

        students.addAll(newStudents);

        System.out.println("After addAll(): " + students);

        // contains(Object o)
        System.out.println("Contains Rahul? "
            + students.contains("Rahul"));

        // containsAll(Collection c)
        System.out.println("Contains Anu and Kiran? "
            + students.containsAll(newStudents));

        // size()
        System.out.println("Size: " + students.size());

        // isEmpty()
        System.out.println("Is collection empty? "
            + students.isEmpty());

        // remove(Object o)
```

```

        students.remove("Arjun");

        System.out.println("After remove(): " + students);

        // removeAll(Collection c)
        students.removeAll(newStudents);

        System.out.println("After removeAll(): " + students);

        // iterator()
        System.out.println("Using iterator():");

        Iterator<String> iterator = students.iterator();

        while (iterator.hasNext()) {
            System.out.println(iterator.next());
        }

        // clear()
        students.clear();

        System.out.println("After clear(): " + students);

        // isEmpty() after clear
        System.out.println("Is collection empty now? "
            + students.isEmpty());
    }
}

```

Compilation & Execution Commands

```

javac CollectionMethodsDemo.java
java CollectionMethodsDemo

```

Output Screenshot

```

PS C:\Users\Student> java CollectionMethodsDemo
After add(): [Rahul, Priya, Arjun]
After addAll(): [Rahul, Priya, Arjun, Anu, Kiran]
Contains Rahul? true
Contains Anu and Kiran? true
Size: 5
Is collection empty? false
After remove(): [Rahul, Priya, Anu, Kiran]
After removeAll(): [Rahul, Priya]
Using iterator():
Rahul
Priya
After clear(): []
Is collection empty now? true
PS C:\Users\Student>

```

Result

The Collection interface methods were successfully demonstrated using an ArrayList.

Program 2: Deque Methods

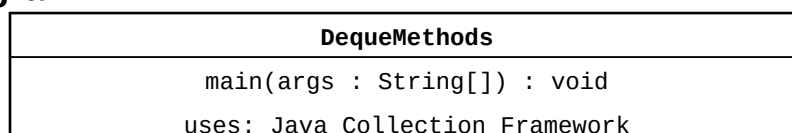
Aim / Objective

To demonstrate insertion, retrieval, and removal operations of a Deque using ArrayDeque.

Algorithm / Logic

1. Create a Deque using ArrayDeque.
2. Insert elements at both ends using addFirst(), addLast(), offerFirst(), and offerLast().
3. View the first and last elements using peekFirst() and peekLast().
4. Remove elements using removeFirst() and removeLast().
5. Remove elements using pollFirst() and pollLast().
6. Display the deque after each operation.

UML Class Diagram



Source Code

```
import java.util.Deque;
import java.util.ArrayDeque;

class DequeMethods {

    public static void main(String[] args) {

        // Creating a Deque
        Deque<String> items = new ArrayDeque<>();

        // addFirst()
        items.addFirst("Book");
        items.addFirst("Pen");

        System.out.println("After addFirst(): " + items);

        // addLast()
        items.addLast("Bag");
        items.addLast("Bottle");

        System.out.println("After addLast(): " + items);

        // offerFirst()
        items.offerFirst("Notebook");

        System.out.println("After offerFirst(): " + items);

        // offerLast()
        items.offerLast("Pencil");

        System.out.println("After offerLast(): " + items);

        // peekFirst()
        System.out.println("First element: " + items.peekFirst());

        // peekLast()
        System.out.println("Last element: " + items.peekLast());

        // removeFirst()
        System.out.println("Removed first: " + items.removeFirst());

        System.out.println("After removeFirst(): " + items);

        // removeLast()
        System.out.println("Removed last: " + items.removeLast());

        System.out.println("After removeLast(): " + items);

        // pollFirst()
        System.out.println("Polled first: " + items.pollFirst());

        System.out.println("After pollFirst(): " + items);

        // pollLast()
        System.out.println("Polled last: " + items.pollLast());

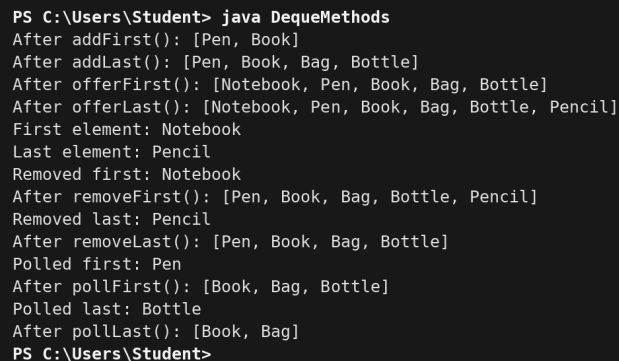
        System.out.println("After pollLast(): " + items);

    }
}
```

Compilation & Execution Commands

```
javac DequeMethods.java
java DequeMethods
```

Output Screenshot



```
PS C:\Users\Student> java DequeMethods
After addFirst(): [Pen, Book]
After addLast(): [Pen, Book, Bag, Bottle]
After offerFirst(): [Notebook, Pen, Book, Bag, Bottle]
After offerLast(): [Notebook, Pen, Book, Bag, Bottle, Pencil]
First element: Notebook
Last element: Pencil
Removed first: Notebook
After removeFirst(): [Pen, Book, Bag, Bottle, Pencil]
Removed last: Pencil
After removeLast(): [Pen, Book, Bag, Bottle]
Polled first: Pen
After pollFirst(): [Book, Bag, Bottle]
Polled last: Bottle
After pollLast(): [Book, Bag]
PS C:\Users\Student>
```

Result

The Deque methods were successfully demonstrated using ArrayDeque.

Program 3: Iterator Methods

Aim / Objective

To demonstrate Iterator methods such as hasNext(), next(), remove(), and forEachRemaining().

Algorithm / Logic

1. Create a List containing student names.
2. Create an Iterator for the list.
3. Traverse using hasNext() and next().
4. Remove Arjun safely using Iterator.remove().
5. Create a new iterator and move to the first element.
6. Process the remaining elements using forEachRemaining().

UML Class Diagram

IteratorMethods
main(args : String[]) : void
uses: Java Collection Framework

Source Code

```
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

class IteratorMethods {

    public static void main(String[] args) {

        // Creating a List
        List<String> students = new ArrayList<>();

        students.add("Rahul");
        students.add("Priya");
        students.add("Arjun");
        students.add("Meera");
        students.add("Anjali");

        System.out.println("Original List: " + students);

        // Creating Iterator
        Iterator<String> iterator = students.iterator();

        // hasNext() and next()
        System.out.println("\nUsing hasNext() and next():");

        while (iterator.hasNext()) {

            String student = iterator.next();

            System.out.println("Student: " + student);

            // remove()
            if (student.equals("Arjun")) {
                iterator.remove();
                System.out.println("Arjun removed.");
            }

        }

        System.out.println("\nList after remove(): " + students);

        // Creating a new Iterator
        iterator = students.iterator();

        // Moving to the first element
        if (iterator.hasNext()) {
            System.out.println("\nFirst element: " + iterator.next());
        }

        // forEachRemaining()
        System.out.println("\nRemaining elements using forEachRemaining():");

        iterator.forEachRemaining(student -> {
```

```

        System.out.println("Student: " + student);
    });
}
}

```

Compilation & Execution Commands

```

javac IteratorMethods.java
java IteratorMethods

```

Output Screenshot

```

PS C:\Users\Student> java IteratorMethods
Original List: [Rahul, Priya, Arjun, Meera, Anjali]

Using hasNext() and next():
Student: Rahul
Student: Priya
Student: Arjun
Arjun removed.
Student: Meera
Student: Anjali

List after remove(): [Rahul, Priya, Meera, Anjali]

First element: Rahul

Remaining elements using forEachRemaining():
Student: Priya
Student: Meera
Student: Anjali
PS C:\Users\Student>

```

Result

The Iterator methods were successfully demonstrated, including safe removal during traversal.

Program 4: ListIterator Methods

Aim / Objective

To demonstrate forward and backward traversal and modification methods of ListIterator.

Algorithm / Logic

1. Create a List of student names.
2. Create a ListIterator.
3. Traverse forward using hasNext(), next(), and nextIndex().
4. Traverse backward using hasPrevious(), previous(), and previousIndex().
5. Use set() to replace an element.
6. Use add() to insert an element.
7. Use remove() to delete an element.

UML Class Diagram

ListIteratorMethods
main(args : String[]) : void
uses: Java Collection Framework

Source Code

```

import java.util.ArrayList;
import java.util.List;
import java.util.ListIterator;

class ListIteratorMethods {

    public static void main(String[] args) {

        // Creating a List
        List<String> students = new ArrayList<>();

        students.add("Rahul");
        students.add("Priya");
        students.add("Arjun");
        students.add("Meera");

        System.out.println("Original List: " + students);

        // Creating ListIterator
        ListIterator<String> iterator = students.listIterator();
    }
}

```

```

// hasNext(), next(), nextIndex()
System.out.println("\nForward Traversal:");

while (iterator.hasNext()) {

    System.out.println(
        "Index: " + iterator.nextIndex() +
        ", Student: " + iterator.next()
    );
}

// hasPrevious(), previous(), previousIndex()
System.out.println("\nBackward Traversal:");

while (iterator.hasPrevious()) {

    System.out.println(
        "Index: " + iterator.previousIndex() +
        ", Student: " + iterator.previous()
    );
}

// Move to first element
iterator = students.listIterator();

// next() and set()
iterator.next();
iterator.set("Aarav");

System.out.println("\nAfter set(): " + students);

// add()
iterator.add("Diya");

System.out.println("After add(): " + students);

// Move to next element and remove it
iterator.next();
iterator.remove();

System.out.println("After remove(): " + students);
}
}

```

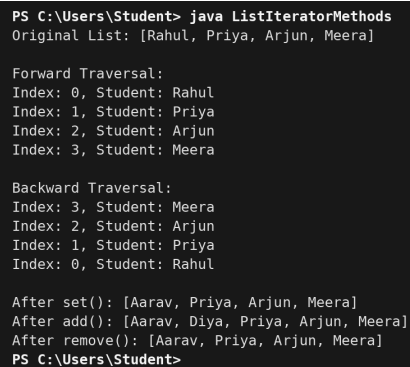
Compilation & Execution Commands

```

javac ListIteratorMethods.java
java ListIteratorMethods

```

Output Screenshot



```

PS C:\Users\Student> java ListIteratorMethods
Original List: [Rahul, Priya, Arjun, Meera]

Forward Traversal:
Index: 0, Student: Rahul
Index: 1, Student: Priya
Index: 2, Student: Arjun
Index: 3, Student: Meera

Backward Traversal:
Index: 3, Student: Meera
Index: 2, Student: Arjun
Index: 1, Student: Priya
Index: 0, Student: Rahul

After set(): [Aarav, Priya, Arjun, Meera]
After add(): [Aarav, Diya, Priya, Arjun, Meera]
After remove(): [Aarav, Priya, Arjun, Meera]
PS C:\Users\Student>

```

Result

The ListIterator methods were successfully demonstrated for bidirectional traversal and list modification.

Program 5: Employee List Demo

Aim / Objective

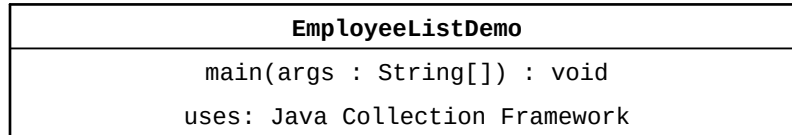
To demonstrate commonly used List methods such as add, get, set, remove, indexOf, lastIndexOf, subList, and sort.

Algorithm / Logic

1. Create an ArrayList of employee names.
2. Add elements and insert one at a specified index.

3. Retrieve and update an element using get() and set().
4. Remove an element using its index.
5. Find first and last occurrences using indexOf() and lastIndexOf().
6. Create a subList.
7. Sort the list in natural order.

UML Class Diagram



Source Code

```
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

class EmployeeListDemo {

    public static void main(String[] args) {

        // Creating a List
        List<String> employees = new ArrayList<>();

        // add(E e)
        employees.add("Anjali");
        employees.add("Vikram");
        employees.add("Rohan");
        employees.add("Anjali");

        System.out.println("Original List: " + employees);

        // add(int index, E element)
        employees.add(1, "Meena");
        System.out.println("After add(1, \"Meena\"): " + employees);

        // get(int index)
        System.out.println("Element at index 2: " + employees.get(2));

        // set(int index, E element)
        employees.set(2, "Karthik");
        System.out.println("After set(2, \"Karthik\"): " + employees);

        // remove(int index)
        employees.remove(3);
        System.out.println("After remove(3): " + employees);

        // indexOf(Object o)
        System.out.println("First index of Anjali: "
            + employees.indexOf("Anjali"));

        // lastIndexOf(Object o)
        System.out.println("Last index of Anjali: "
            + employees.lastIndexOf("Anjali"));

        // subList(int fromIndex, int toIndex)
        List<String> selectedEmployees = employees.subList(1, 3);
        System.out.println("SubList from index 1 to 3: "
            + selectedEmployees);

        // sort(Comparator)
        employees.sort(Comparator.naturalOrder());
        System.out.println("After Sorting: " + employees);
    }
}
```

Compilation & Execution Commands

```
javac EmployeeListDemo.java
java EmployeeListDemo
```

Output Screenshot

```

PS C:\Users\Student> java EmployeeListDemo
Original List: [Anjali, Vikram, Rohan, Anjali]
After add(1, "Meena"): [Anjali, Meena, Vikram, Rohan, Anjali]
Element at index 2: Vikram
After set(2, "Karthik"): [Anjali, Meena, Karthik, Rohan, Anjali]
After remove(3): [Anjali, Meena, Karthik, Anjali]
First index of Anjali: 0
Last index of Anjali: 3
SubList from index 1 to 3: [Meena, Karthik]
After Sorting: [Anjali, Anjali, Karthik, Meena]
PS C:\Users\Student>

```

Result

The List operations were successfully demonstrated using an ArrayList.

Program 6: Map Methods

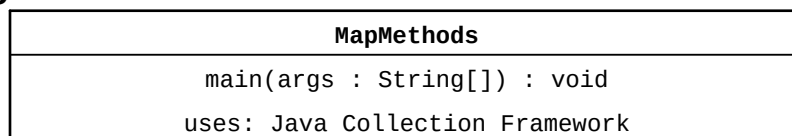
Aim / Objective

To demonstrate commonly used methods of the Map interface using HashMap.

Algorithm / Logic

1. Create a HashMap of student IDs and names.
2. Add key-value pairs using put().
3. Retrieve and search values using get(), containsKey(), and containsValue().
4. Display keys, values, and entries.
5. Check size and empty status.
6. Remove one entry and clear the map.
7. Check the empty status after clearing.

UML Class Diagram



Source Code

```

import java.util.Map;
import java.util.HashMap;

class MapMethods {

    public static void main(String[] args) {

        // Creating a Map
        Map<Integer, String> students = new HashMap<>();

        // put()
        students.put(101, "Aarav");
        students.put(102, "Meera");
        students.put(103, "Rohan");
        students.put(104, "Diya");

        System.out.println("Map after put(): " + students);

        // get()
        System.out.println("Value for key 102: " + students.get(102));

        // containsKey()
        System.out.println("Contains key 103: "
            + students.containsKey(103));

        // containsValue()
        System.out.println("Contains value Rohan: "
            + students.containsValue("Rohan"));

        // keySet()
        System.out.println("Keys: " + students.keySet());
    }
}

```

```

        // values()
        System.out.println("Values: " + students.values());

        // entrySet()
        System.out.println("Entries: " + students.entrySet());

        // size()
        System.out.println("Map size: " + students.size());

        // isEmpty()
        System.out.println("Is map empty: " + students.isEmpty());

        // remove()
        students.remove(104);
        System.out.println("After remove(104): " + students);

        // clear()
        students.clear();
        System.out.println("After clear(): " + students);

        // isEmpty() after clear
        System.out.println("Is map empty after clear: "
            + students.isEmpty());
    }
}

```

Compilation & Execution Commands

```

javac MapMethods.java
java MapMethods

```

Output Screenshot

```

PS C:\Users\Student> java MapMethods
Map after put(): {101=Aarav, 102=Meera, 103=Rohan, 104=Diya}
Value for key 102: Meera
Contains key 103: true
Contains value Rohan: true
Keys: [101, 102, 103, 104]
Values: [Aarav, Meera, Rohan, Diya]
Entries: [101=Aarav, 102=Meera, 103=Rohan, 104=Diya]
Map size: 4
Is map empty: false
After remove(104): {101=Aarav, 102=Meera, 103=Rohan}
After clear(): {}
Is map empty after clear: true
PS C:\Users\Student>

```

Result

The Map interface methods were successfully demonstrated using HashMap.

Program 7: NavigableMap Methods Demo

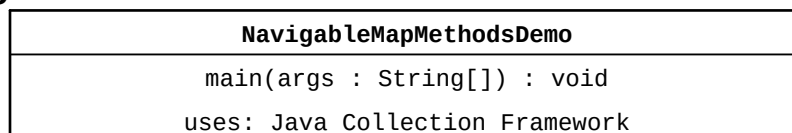
Aim / Objective

To demonstrate navigation operations of NavigableMap using TreeMap.

Algorithm / Logic

1. Create a NavigableMap using TreeMap.
2. Insert sorted key-value pairs.
3. Use lowerKey(), floorKey(), ceilingKey(), and higherKey().
4. Display firstEntry() and lastEntry().
5. Remove boundary entries using pollFirstEntry() and pollLastEntry().
6. Display the map in descending order.

UML Class Diagram



Source Code

```

import java.util.NavigableMap;

```

```

import java.util.TreeMap;

class NavigableMapMethodsDemo {

    public static void main(String[] args) {

        // Creating a NavigableMap using TreeMap
        NavigableMap<Integer, String> students = new TreeMap<>();

        // Adding key-value pairs
        students.put(101, "Aarav");
        students.put(103, "Rohan");
        students.put(105, "Diya");
        students.put(107, "Anjali");
        students.put(109, "Meera");

        System.out.println("NavigableMap: " + students);

        // lowerKey()
        System.out.println("Lower Key of 105: " + students.lowerKey(105));

        // floorKey()
        System.out.println("Floor Key of 105: " + students.floorKey(105));

        // ceilingKey()
        System.out.println("Ceiling Key of 106: " + students.ceilingKey(106));

        // higherKey()
        System.out.println("Higher Key of 105: " + students.higherKey(105));

        // firstEntry()
        System.out.println("First Entry: " + students.firstEntry());

        // lastEntry()
        System.out.println("Last Entry: " + students.lastEntry());

        // pollFirstEntry()
        System.out.println("Poll First Entry: " + students.pollFirstEntry());
        System.out.println("After pollFirstEntry(): " + students);

        // pollLastEntry()
        System.out.println("Poll Last Entry: " + students.pollLastEntry());
        System.out.println("After pollLastEntry(): " + students);

        // descendingMap()
        System.out.println("Descending Map: " + students.descendingMap());

    }
}

```

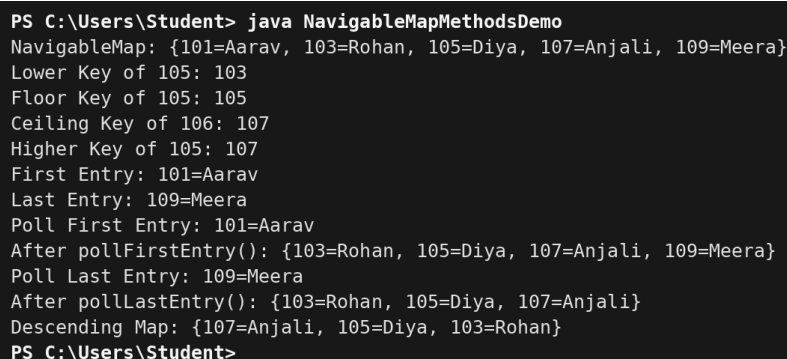
Compilation & Execution Commands

```

javac NavigableMapMethodsDemo.java
java NavigableMapMethodsDemo

```

Output Screenshot



```

PS C:\Users\Student> java NavigableMapMethodsDemo
NavigableMap: {101=Aarav, 103=Rohan, 105=Diya, 107=Anjali, 109=Meera}
Lower Key of 105: 103
Floor Key of 105: 105
Ceiling Key of 106: 107
Higher Key of 105: 107
First Entry: 101=Aarav
Last Entry: 109=Meera
Poll First Entry: 101=Aarav
After pollFirstEntry(): {103=Rohan, 105=Diya, 107=Anjali, 109=Meera}
Poll Last Entry: 109=Meera
After pollLastEntry(): {103=Rohan, 105=Diya, 107=Anjali}
Descending Map: {107=Anjali, 105=Diya, 103=Rohan}
PS C:\Users\Student>

```

Result

The NavigableMap methods were successfully demonstrated using TreeMap.

Program 8: NavigableSet Methods

Aim / Objective

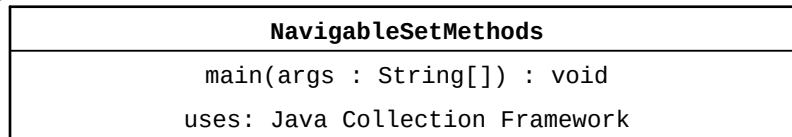
To demonstrate navigation and boundary-removal methods of NavigableSet using TreeSet.

Algorithm / Logic

1. Create a NavigableSet using TreeSet.

2. Add integer values to the set.
3. Use lower(), floor(), ceiling(), and higher().
4. Remove the first and last elements using pollFirst() and pollLast().
5. Display the set in descending order using descendingSet().

UML Class Diagram



Source Code

```
import java.util.NavigableSet;
import java.util.TreeSet;

class NavigableSetMethods {

    public static void main(String[] args) {

        // Creating a NavigableSet using TreeSet
        NavigableSet<Integer> values = new TreeSet<>();

        // Adding elements
        values.add(10);
        values.add(20);
        values.add(30);
        values.add(40);
        values.add(50);
        values.add(60);

        System.out.println("NavigableSet: " + values);

        // lower()
        System.out.println("Lower than 30: " + values.lower(30));

        // floor()
        System.out.println("Floor of 30: " + values.floor(30));

        // ceiling()
        System.out.println("Ceiling of 35: " + values.ceiling(35));

        // higher()
        System.out.println("Higher than 30: " + values.higher(30));

        // pollFirst()
        System.out.println("Removed first element: " + values.pollFirst());

        System.out.println("After pollFirst(): " + values);

        // pollLast()
        System.out.println("Removed last element: " + values.pollLast());

        System.out.println("After pollLast(): " + values);

        // descendingSet()
        System.out.println("Descending Set: " + values.descendingSet());

    }
}
```

Compilation & Execution Commands

```
javac NavigableSetMethods.java
java NavigableSetMethods
```

Output Screenshot

```

PS C:\Users\Student> java NavigableSetMethods
NavigableSet: [10, 20, 30, 40, 50, 60]
Lower than 30: 20
Floor of 30: 30
Ceiling of 35: 40
Higher than 30: 40
Removed first element: 10
After pollFirst(): [20, 30, 40, 50, 60]
Removed last element: 60
After pollLast(): [20, 30, 40, 50]
Descending Set: [50, 40, 30, 20]
PS C:\Users\Student>

```

Result

The NavigableSet methods were successfully demonstrated using TreeSet.

Program 9: Queue Method

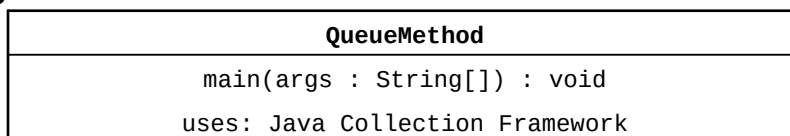
Aim / Objective

To demonstrate common Queue operations using a LinkedList implementation.

Algorithm / Logic

1. Create a Queue using LinkedList.
2. Add tasks using add().
3. Insert another task using offer().
4. View the head using element() and peek().
5. Remove the head using remove().
6. Remove the next head using poll().
7. Display the queue after each operation.

UML Class Diagram



Source Code

```

import java.util.Queue;
import java.util.LinkedList;

class QueueMethod {

    public static void main(String[] args) {

        // Creating a Queue
        Queue<String> tasks = new LinkedList<>();

        // add()
        tasks.add("Study");
        tasks.add("Practice");
        tasks.add("Assignment");

        System.out.println("Queue after add(): " + tasks);

        // offer()
        tasks.offer("Project");

        System.out.println("Queue after offer(): " + tasks);

        // element()
        System.out.println("Head using element(): " + tasks.element());

        // peek()
        System.out.println("Head using peek(): " + tasks.peek());

        // remove()
        System.out.println("Removed using remove(): " + tasks.remove());
        System.out.println("Queue after remove(): " + tasks);
    }
}

```

```

        // poll()
        System.out.println("Removed using poll(): " + tasks.poll());

        System.out.println("Queue after poll(): " + tasks);
    }
}

```

Compilation & Execution Commands

```

javac QueueMethod.java
java QueueMethod

```

Output Screenshot

```

PS C:\Users\Student> java QueueMethod
Queue after add(): [Study, Practice, Assignment]
Queue after offer(): [Study, Practice, Assignment, Project]
Head using element(): Study
Head using peek(): Study
Removed using remove(): Study
Queue after remove(): [Practice, Assignment, Project]
Removed using poll(): Practice
Queue after poll(): [Assignment, Project]
PS C:\Users\Student>

```

Result

The Queue methods were successfully demonstrated using LinkedList.

Program 10: Set Method

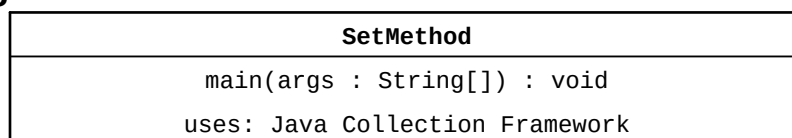
Aim / Objective

To demonstrate basic Set operations and the handling of duplicate elements using HashSet.

Algorithm / Logic

1. Create a HashSet of course names.
2. Add Java, Python, C, and a duplicate Java.
3. Display the set and observe that the duplicate is not added.
4. Remove C and check whether Python exists.
5. Display the size and empty status.
6. Clear the set and check the empty status again.

UML Class Diagram



Source Code

```

import java.util.HashSet;
import java.util.Set;

class SetMethod {

    public static void main(String[] args) {

        // Creating a Set
        Set<String> courses = new HashSet<>();

        // add(E e)
        courses.add("Java");
        courses.add("Python");
        courses.add("C");
        courses.add("Java");    // Duplicate value is not added

        System.out.println("After add(): " + courses);

        // remove(Object o)
        courses.remove("C");
    }
}

```

```

        System.out.println("After remove(): " + courses);

        // contains(Object o)
        System.out.println("Contains Python? " + courses.contains("Python"));

        // size()
        System.out.println("Set size: " + courses.size());

        // isEmpty()
        System.out.println("Is set empty? " + courses.isEmpty());

        // clear()
        courses.clear();
        System.out.println("After clear(): " + courses);

        // Checking again
        System.out.println("Is set empty after clear()? " + courses.isEmpty());
    }
}

```

Compilation & Execution Commands

```

javac SetMethod.java
java SetMethod

```

Output Screenshot

```

PS C:\Users\Student> java SetMethod
After add(): [Java, C, Python]
After remove(): [Java, Python]
Contains Python? true
Set size: 2
Is set empty? false
After clear(): []
Is set empty after clear()? true
PS C:\Users\Student>

```

Result

The Set operations were successfully demonstrated using HashSet, including duplicate prevention.

Program 11: SortedMap Method

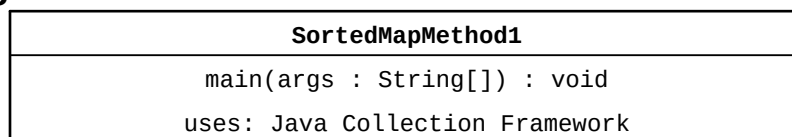
Aim / Objective

To demonstrate SortedMap operations using TreeMap and key-based ordering.

Algorithm / Logic

1. Create a SortedMap using TreeMap.
2. Add key-value pairs in unsorted order.
3. Display the map in sorted key order.
4. Retrieve the first and last keys.
5. Display headMap(), tailMap(), and subMap() ranges.
6. Display the comparator.

UML Class Diagram



Source Code

```

import java.util.SortedMap;
import java.util.TreeMap;

class SortedMapMethod1 {

    public static void main(String[] args) {
        // Creating a SortedMap using TreeMap
    }
}

```

```

SortedMap<Integer, String> students = new TreeMap<>();

// Adding key-value pairs
students.put(103, "Rohan");
students.put(101, "Aarav");
students.put(105, "Diya");
students.put(102, "Meera");
students.put(104, "Anjali");

System.out.println("SortedMap: " + students);

// firstKey()
System.out.println("First Key: " + students.firstKey());

// lastKey()
System.out.println("Last Key: " + students.lastKey());

// headMap()
System.out.println("HeadMap (< 103): " + students.headMap(103));

// tailMap()
System.out.println("TailMap (>= 103): " + students.tailMap(103));

// subMap()
System.out.println("SubMap (102 to 105): " + students.subMap(102, 105));

// comparator()
System.out.println("Comparator: " + students.comparator());
}
}

```

Compilation & Execution Commands

```

javac SortedMapMethod1.java
java SortedMapMethod1

```

Output Screenshot

```

PS C:\Users\Student> java SortedMapMethod1
SortedMap: {101=Aarav, 102=Meera, 103=Rohan, 104=Anjali, 105=Diya}
First Key: 101
Last Key: 105
HeadMap (< 103): {101=Aarav, 102=Meera}
TailMap (>= 103): {103=Rohan, 104=Anjali, 105=Diya}
SubMap (102 to 105): {102=Meera, 103=Rohan, 104=Anjali}
Comparator: null
PS C:\Users\Student>

```

Result

The SortedMap methods were successfully demonstrated using TreeMap.

Program 12: SortedSet Method

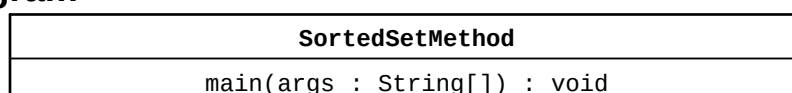
Aim / Objective

To demonstrate SortedSet operations using TreeSet and automatic element ordering.

Algorithm / Logic

1. Create a SortedSet using TreeSet.
2. Add integer elements in unsorted order.
3. Display the set in sorted order.
4. Retrieve the first and last elements.
5. Display headSet(), tailSet(), and subSet() ranges.
6. Display the comparator.

UML Class Diagram



Source Code

```
import java.util.SortedSet;
import java.util.TreeSet;

class SortedSetMethod {

    public static void main(String[] args) {

        // Creating a SortedSet using TreeSet
        SortedSet<Integer> numbers = new TreeSet<>();

        // Adding elements
        numbers.add(40);
        numbers.add(10);
        numbers.add(70);
        numbers.add(20);
        numbers.add(50);
        numbers.add(30);

        System.out.println("SortedSet: " + numbers);

        // first()
        System.out.println("First element: " + numbers.first());

        // last()
        System.out.println("Last element: " + numbers.last());

        // headSet()
        System.out.println("HeadSet before 40: " + numbers.headSet(40));

        // tailSet()
        System.out.println("TailSet from 30: " + numbers.tailSet(30));

        // subSet()
        System.out.println("SubSet from 20 to 60: " + numbers.subSet(20, 60));

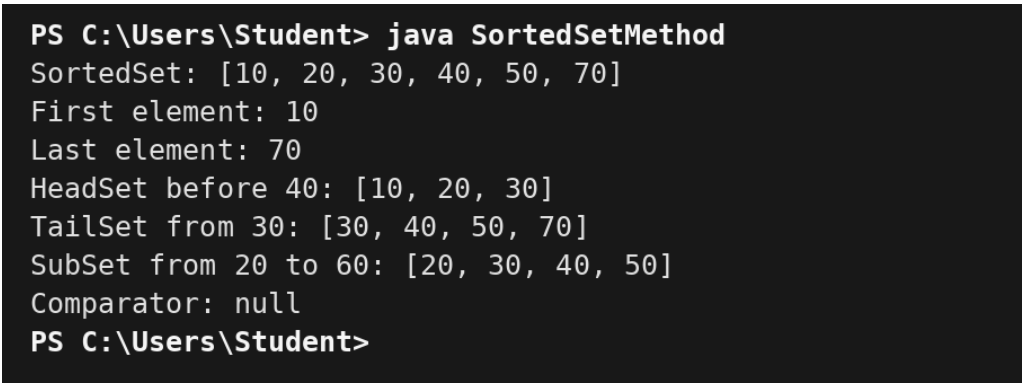
        // comparator()
        System.out.println("Comparator: " + numbers.comparator());

    }
}
```

Compilation & Execution Commands

```
javac SortedSetMethod.java
java SortedSetMethod
```

Output Screenshot



```
PS C:\Users\Student> java SortedSetMethod
SortedSet: [10, 20, 30, 40, 50, 70]
First element: 10
Last element: 70
HeadSet before 40: [10, 20, 30]
TailSet from 30: [30, 40, 50, 70]
SubSet from 20 to 60: [20, 30, 40, 50]
Comparator: null
PS C:\Users\Student>
```

Result

The SortedSet methods were successfully demonstrated using TreeSet.